

Biconoirs Gourmet - Backend Class Operation

Postman testing guide for all current URIs of the NestJS + Prisma + Supabase project.

Base Configuration

Recommended Postman variable:
baseUrl = http://localhost:3000/ops

When deploying on AWS:
baseUrl = http://PUBLIC_IP_EC2:3000/ops
or with a domain:
baseUrl = https://YOUR_DOMAIN/ops

Common headers:

Content-Type: application/json

For cart and Auth /me:
x-user-id: {{userId}}

For inventory:
x-user-id: {{adminUserId}}

For logout:
authorization: {{refreshToken}}

Suggested variables in Postman:

baseUrl
userId
adminUserId
customerId
dishId
categoryId
orderId
inventoryId
skuCode
refreshToken
resetToken

Auth

POST /auth/register

Create user.

Method	POST
URL	{{baseUrl}}/auth/register

Body JSON:

```
{
  "name": "Juan Perez",
  "email": "juan@test.com",
  "passwordHash": "clave12345",
```

```
"phone": "0999999999"
}
```

POST /auth/login

Search for user by email according to the current service.

Method	POST
URL	{{baseUrl}}/auth/login

Body JSON:

```
{
  "email": "juan@test.com",
  "password": "key12345"
}
```

POST /auth/refresh

Check session by refreshing token.

Method	POST
URL	{{baseUrl}}/auth/refresh

Body JSON:

```
{
  "refreshToken": "{{refreshToken}}"
}
```

POST /auth/logout

Revoke refresh token.

Method	POST
URL	{{baseUrl}}/auth/logout

Headers:

```
authorization: {{refreshToken}}
```

Body: without body.

GET /auth/me

View current user data.

Method	GET
URL	{{baseUrl}}/auth/me

Headers:

```
x-user-id: {{userId}}
```

Body: without body.

POST /auth/password/forgot

Search for user by email for recovery.

Method	POST
URL	{{baseUrl}}/auth/password/forgot

Body JSON:

```
{
  "email": "juan@test.com"
}
```

POST /auth/password/reset

Validate current recovery token.

Method	POST
URL	{{baseUrl}}/auth/password/reset

Body JSON:

```
{
  "token": "{{resetToken}}",
  "newPassword": "newClave123"
}
```

Customers

PUT /customers/{{customerId}}

Update customer data.

Method	PUT
URL	{{baseUrl}}/customers/{{customerId}}

Body JSON:

```
{
  "name": "Juan Perez Actualizado",
  "email": "juan.actualizado@test.com",
  "phone": "0988888888"
}
```

GET /customers/{{customerId}}/orders

List a customer's orders.

Method	GET
URL	{{baseUrl}}/customers/{{customerId}}/orders

Body: without body.

PATCH /customers/{{customerId}}/preferences

Update customer preferences.

Method	PATCH
URL	{{baseUrl}}/customers/{{customerId}}/preferences

Body JSON:

```
{
  "preferences": {
    "favoriteCategory": "desserts",
    "notifications": true,
    "spicyLevel": "medium"
  }
}
```

Menu

GET /menu/dishes

List dishes.

Method	GET
URL	{{baseUrl}}/menu/dishes

Body: without body.

GET /menu/dishes/{{dishId}}

Get a detailed look at a dish.

Method	GET
URL	{{baseUrl}}/menu/dishes/{{dishId}}

Body: without body.

POST /menu/dishes

Create dish. The current service generates UUIDs internally even though the DTO declares itemId.

Method	POST
URL	{{baseUrl}}/menu/dishes

Body JSON:

```
{
  "itemId": "plato-001",
  "name": "Meat Lasana",
  "description": "Artisanal lasana with bolognese sauce",
  "price": 9.5,
  "categoryId": "{{categoryId}}",
  "imageUrl": "https://example.com/lasana.jpg",
  "isAvailable": true,
  "availabilityReason": null
}
```

PUT /menu/dishes/{{dishId}}

Update dish.

Method	PUT
URL	{{baseUrl}}/menu/dishes/{{dishId}}

Body JSON:

```
{
  "name": "Pasta Alfredo",
  "description": "Pasta with creamy sauce",
  "price": 8.5,
  "categoryId": "{{categoryId}}",
  "imageUrl": "https://example.com/pasta.jpg",
  "isAvailable": true,
  "availabilityReason": null
}
```

DELETE /menu/dishes/{{dishId}}

Remove dish.

Method	DELETE
--------	--------

URL	{{baseUrl}}/menu/dishes/{{dishId}}
-----	------------------------------------

Body: without body.

PATCH /menu/dishes/{{dishId}}/availability

Update dish availability.

Method	PATCH
URL	{{baseUrl}}/menu/dishes/{{dishId}}/availability

Body JSON:

```
{
  "isAvailable": false,
  "availabilityReason": "Temporarily out of stock"
}
```

POST /menu/dishes/{{dishId}}/ingredients

Add ingredient to a dish.

Method	POST
URL	{{baseUrl}}/menu/dishes/{{dishId}}/ingredients

Body JSON:

```
{
  "skuCode": "{{skuCode}}",
  "quantityRequired": 2
}
```

DELETE /menu/dishes/{{dishId}}/ingredients/{{skuCode}}

Remove ingredient associated with a dish.

Method	DELETE
URL	{{baseUrl}}/menu/dishes/{{dishId}}/ingredients/{{skuCode}}

Body: without body.

GET /menu/categories

List categories.

Method	GET
URL	{{baseUrl}}/menu/categories

Body: without body.

POST /menu/categories

Create category. The current service generates UUIDs internally even though the DTO declares categoryId.

Method	POST
URL	{{baseUrl}}/menu/categories

Body JSON:

```
{
  "categoryId": "cat-001",
  "name": "Pastas",
  "description": "Main dishes with pasta",
  "sortOrder": 1,
  "active": true
}
```

PUT /menu/categories/{{categoryId}}

Update category.

Method	PUT
URL	{{baseUrl}}/menu/categories/{{categoryId}}

Body JSON:

```
{
  "name": "Entradas",
  "description": "Platos para empezar",
  "sortOrder": 1,
  "active": true
}
```

GET /menu/ingredients

List ingredients from the menu module.

Method	GET
URL	{{baseUrl}}/menu/ingredients

Body: without body.

Orders

GET /orders

List orders.

Method	GET
URL	{{baseUrl}}/orders

Body: without body.

GET /orders/{{orderId}}

Order details.

Method	GET
URL	{{baseUrl}}/orders/{{orderId}}

Body: without body.

POST /orders

Create order. The current service generates UUIDs internally even though the DTO declares orderId.

Method	POST
URL	{{baseUrl}}/orders

Body JSON:

```
{
  "orderId": "orden-001",
  "userId": "{{userId}}",
  "totalAmount": 19,
  "status": "pending",
  "deliveryType": "delivery",
  "deliveryAddress": "Av. Principal 123",
  "specialInstructions": "No onion",
  "items": [
    {
```

```
"itemId": "{{dishId}}",
"quantity": 2,
"priceAtPurchase": 9.5
}
]
```

PUT /orders/{{orderId}}

Update order.

Method	PUT
URL	{{baseUrl}}/orders/{{orderId}}

Body JSON:

```
{
"totalAmount": 25.75,
"status": "confirmed",
"deliveryType": "delivery",
"deliveryAddress": "Av. Principal 123",
"specialInstructions": "No onion"
}
```

DELETE /orders/{{orderId}}

Delete order.

Method	DELETE
URL	{{baseUrl}}/orders/{{orderId}}

Body: without body.

Cart

GET /cart

View user cart.

Method	GET
URL	{{baseUrl}}/cart

Headers:

```
x-user-id: {{userId}}
```

Body: without body.

POST /cart/items

Add dish to cart.

Method	POST
URL	{{baseUrl}}/cart/items

Headers:

```
x-user-id: {{userId}}
```

Body JSON:

```
{
"dish_id": "{{dishId}}",
```

```
"quantity": 2
}
```

PUT /cart/items/{{dishId}}

Update the quantity of an item in the cart.

Method	PUT
URL	{{baseUrl}}/cart/items/{{dishId}}

Headers:

```
x-user-id: {{userId}}
```

Body JSON:

```
{
  "quantity": 3
}
```

PATCH /cart/items/{{dishId}}

Partially update the quantity of an item in the shopping cart.

Method	PATCH
URL	{{baseUrl}}/cart/items/{{dishId}}

Headers:

```
x-user-id: {{userId}}
```

Body JSON:

```
{
  "quantity": 1
}
```

DELETE /cart/items/{{dishId}}

Remove an item from the cart.

Method	DELETE
URL	{{baseUrl}}/cart/items/{{dishId}}

Headers:

```
x-user-id: {{userId}}
```

Body: without body.

DELETE /cart

Clean the entire cart.

Method	DELETE
URL	{{baseUrl}}/cart

Headers:

```
x-user-id: {{userId}}
```

Body: without body.

Inventory

GET /inventory

List inventory. Requires admin.

Method	GET
URL	{{baseUrl}}/inventory

Headers:

```
x-user-id: {{adminUserId}}
```

Body: without body.

GET /inventory/{{inventoryId}}

Inventory details. Requires admin.

Method	GET
URL	{{baseUrl}}/inventory/{{inventoryId}}

Headers:

```
x-user-id: {{adminUserId}}
```

Body: without body.

POST /inventory

Create inventory item and associated ingredient. Requires admin.

Method	POST
URL	{{baseUrl}}/inventory

Headers:

```
x-user-id: {{adminUserId}}
```

Body JSON:

```
{
  "ingredient_name": "Tomato",
  "current_stock": 25,
  "unit": "kg",
  "reorder_level": 5,
  "supplier": "Local Supplier",
  "unit_cost": 1.5,
  "expiry_date": "2026-07-15"
}
```

PUT /inventory/{{inventoryId}}

Update inventory item. Requires admin.

Method	PUT
URL	{{baseUrl}}/inventory/{{inventoryId}}

Headers:

```
x-user-id: {{adminUserId}}
```

Body JSON:

```
{
  "current_stock": 40,
  "reorder_level": 10,
  "expiry_date": "2026-08-20"
}
```

PATCH /inventory/{{inventoryId}}

Partially update inventory item. Requires admin.

Method	PATCH
URL	{{baseUrl}}/inventory/{{inventoryId}}

Headers:

```
x-user-id: {{adminUserId}}
```

Body JSON:

```
{
  "current_stock": 15
}
```

DELETE /inventory/{{inventoryId}}

Remove item from inventory. Requires admin.

Method	DELETE
URL	{{baseUrl}}/inventory/{{inventoryId}}

Headers:

```
x-user-id: {{adminUserId}}
```

Body: without body.

Ingredients

GET /ingredients

Public catalog of ingredients.

Method	GET
URL	{{baseUrl}}/ingredients

Body: without body.

GET /ingredients?name=tomato

Filter ingredients by name.

Method	GET
URL	{{baseUrl}}/ingredients?name=tomato

Body: without body.

GET /ingredients?category=vegetables

Filter ingredients by category.

Method	GET
URL	{{baseUrl}}/ingredients?category=vegetables

Body: without body.

Recommended test order

1. GET /ingredients
2. POST /auth/register
3. GET /menu/categories
4. POST /menu/categories
5. POST /menu/dishes
6. GET /menu/dishes and copy real dishId
7. POST /cart/items with x-user-id
8. PUT/PATCH/DELETE /cart/items/{{dishId}}
9.
POST
/ inventory with
admin's

Important notes

If a protected route returns 401, it is usually because the x-user-id header is missing or the user does not exist in Supabase.

If inventory responds 403, the user exists but does not have the admin role.

If it returns 404, check that you are using the /ops prefix and that the ID exists.

For AWS, replace baseUrl with the configured public IP or domain.